

Техническое задание: HH Vacancy Analytics Platform

Сбор и анализ открытых вакансий через официальный HeadHunter API с
локальной AI-фильтрацией

Учебно-портфолио проект

2026-06-04

Содержание

1. Назначение документа	3
2. Краткое описание проекта	3
3. Актуальные внешние ограничения	3
3.1. Официальный API	3
3.2. Разделение методов API	4
3.3. Границы безопасности проекта	4
4. Цели проекта	4
4.1. Бизнес-цель	4
4.2. Учебная цель	4
5. Пользователи системы	5
5.1. Основной пользователь	5
5.2. Разработчик проекта	5
6. Область проекта	5
6.1. Входит в MVP	5
6.2. Не входит в MVP	5
7. Основные пользовательские сценарии	6
7.1. Создание поисковой конфигурации	6
7.2. Ручной запуск сбора вакансий	6
7.3. Просмотр аналитики	6
7.4. AI-фильтрация вакансий	7
7.5. Подготовка ручного отклика	7
8. Функциональные требования	7
8.1. Модуль HH API Client	7
8.2. Модуль поисковых конфигураций	8
8.3. Модуль сбора вакансий	8
8.4. Модуль хранения raw JSON	8
8.5. Модуль нормализации	9
8.6. Модуль навыков	9
8.7. Модуль аналитики	9
8.8. Модуль профиля пользователя	10
8.9. Модуль rule-based скоринга	10
8.10. Модуль локальной AI-фильтрации	10
8.11. Модуль генерации черновика отклика	11

9. Нефункциональные требования	11
9.1. Надёжность	11
9.2. Производительность MVP	11
9.3. Масштабируемость	11
9.4. Безопасность	12
9.5. Логирование	12
10. Архитектура системы	12
10.1. MVP-архитектура	12
10.2. Архитектура с фоновыми задачами	12
11. Рекомендуемый стек	13
11.1. Backend	13
11.2. База данных	13
11.3. AI-модуль	13
11.4. Инфраструктура	13
12. Структура проекта	13
13. Модель данных	15
13.1. search_configs	15
13.2. vacancies	15
13.3. skills	16
13.4. vacancy_skills	16
13.5. parser_runs	16
13.6. raw_vacancy_payloads	16
13.7. user_profiles	16
13.8. vacancy_scores	17
13.9. cover_letter_drafts	17
13.10. application_decisions	17
14. REST API	17
14.1. Health	17
14.2. Search configs	17
14.3. Parser	18
14.4. Vacancies	18
14.5. Analytics	18
14.6. Profiles	18
14.7. Recommendations	18
14.8. Drafts	19
15. Алгоритм сбора вакансий	19
16. Алгоритм рекомендации вакансий	19
17. Критерии приёмки MVP	19
18. Этапы реализации	20
Этап 1. Минимальный HH API client	20
Этап 2. Сохранение raw JSON	20
Этап 3. PostgreSQL и модели	20
Этап 4. Дедупликация	20
Этап 5. FastAPI	20
Этап 6. Аналитика	20
Этап 7. Профиль пользователя и rule-based scoring	21
Этап 8. Локальная AI-фильтрация	21
Этап 9. Черновики откликов	21
Этап 10. Docker и документация	21

19. Риски и меры снижения	21
20. Документы проекта	22
21. Формулировка для резюме	22
22. Источники	22

1. Назначение документа

Документ описывает техническое задание на разработку проекта **HH Vacancy Analytics Platform** - учебно-портфолио платформы для сбора, хранения, анализа и интеллектуальной фильтрации открытых вакансий HeadHunter через официальный API.

Проект не является ботом для массовых откликов, не автоматизирует действия пользователя на сайте HeadHunter, не собирает резюме соискателей и не обходит ограничения HeadHunter. Финальное решение об отклике принимает человек, а отклик выполняется вручную через интерфейс HeadHunter.

2. Краткое описание проекта

HH Vacancy Analytics Platform - backend-платформа, которая:

- получает открытые вакансии через официальный HeadHunter API;
- сохраняет исходные JSON-ответы и нормализованные данные;
- анализирует вакансии по навыкам, зарплатам, городам, работодателям и опыту;
- сравнивает вакансии с профилем пользователя;
- рассчитывает оценку релевантности вакансии;
- объясняет, почему вакансия подходит или не подходит;
- готовит черновик сопроводительного письма;
- оставляет финальное решение об отклике пользователю.

Ключевой пайплайн проекта:

```
HH API
-----> Python API Client
-----> Vacancy Collector
-----> Raw JSON Storage
-----> Normalizer
-----> PostgreSQL
-----> Analytics Service
-----> Local AI Scoring Service
-----> Recommendations
-----> Cover Letter Draft
-----> Manual Apply by User
```

3. Актуальные внешние ограничения

3.1. Официальный API

В официальной документации HeadHunter API указано, что API работает по HTTPS, использует JSON, имеет базовый URL <https://api.hh.ru/>, а авторизация выполняется через OAuth2 для методов, которым она требуется. В документации также присутствуют методы работы с вакансиями, включая поиск и просмотр вакансий.

3.2. Разделение методов API

Документация HeadHunter API различает несколько типов методов:

- anonymous - анонимные запросы, не требуют авторизации;
- client - запросы от имени приложения;
- applicant - запросы от имени соискателя;
- employer - методы работодателя.

Для данного проекта используются только открытые данные вакансий и методы, не требующие действий от имени соискателя.

3.3. Границы безопасности проекта

Проект не должен:

- автоматически отправлять отклики;
- собирать резюме соискателей;
- обрабатывать персональные данные кандидатов;
- использовать cookies пользователя для имитации браузера;
- обходить ограничения API;
- скрейпить закрытые страницы;
- нарушать условия использования HeadHunter.

Проект должен быть сформулирован как **аналитическая и рекомендательная система**, а не как автокликер или бот для массовых откликов.

4. Цели проекта

4.1. Бизнес-цель

Создать систему, которая помогает пользователю анализировать рынок вакансий и быстрее находить подходящие позиции.

Система должна отвечать на вопросы:

- какие навыки чаще всего требуют работодатели;
- какие зарплаты предлагают по разным направлениям;
- какие города и компании наиболее активны;
- какие вакансии лучше всего подходят под профиль пользователя;
- какие навыки отсутствуют у пользователя относительно требований рынка;
- по каким вакансиям стоит подготовить ручной отклик.

4.2. Учебная цель

Проект должен развивать навыки:

- Python backend-разработки;
- работы с HTTP API;
- обработки JSON;
- проектирования PostgreSQL-схемы;
- ETL-процессов;
- FastAPI;
- SQLAlchemy и Alembic;
- фоновых задач и планировщиков;
- Docker и Docker Compose;

- аналитики данных;
- локального AI-инференса;
- построения рекомендательных систем.

5. Пользователи системы

5.1. Основной пользователь

Пользователь, который ищет работу или анализирует рынок труда.

Его задачи:

- настроить поисковые запросы;
- получить список вакансий;
- увидеть аналитику рынка;
- получить рейтинг подходящих вакансий;
- понять, почему вакансия подходит или не подходит;
- сформировать черновик сопроводительного письма;
- вручную принять решение об отклике.

5.2. Разработчик проекта

Разработчик поддерживает систему, добавляет новые источники данных, улучшает скоринг, настраивает инфраструктуру и расширяет аналитику.

6. Область проекта

6.1. Входит в MVP

В MVP входит:

1. Получение вакансий через официальный HeadHunter API.
2. Настройка поисковых запросов.
3. Пагинация результатов.
4. Сохранение raw JSON.
5. Нормализация вакансий.
6. Сохранение вакансий в PostgreSQL.
7. Дедупликация по hh_id вакансии.
8. Сохранение навыков и связей вакансий с навыками.
9. Логирование запусков парсера.
10. REST API для просмотра вакансий.
11. Базовая аналитика по вакансиям.
12. Rule-based скоринг вакансий.
13. Локальная AI-оценка вакансий.
14. Генерация черновика сопроводительного письма.
15. Docker Compose для локального запуска.

6.2. Не входит в MVP

В MVP не входит:

1. Автоматическая отправка откликов.
2. Массовые отклики.
3. Работа с резюме через applicant API.

4. Сбор персональных данных соискателей.
5. Скрейпинг закрытых страниц.
6. Обход антибот-защиты.
7. Использование браузерной автоматизации для имитации действий пользователя.
8. Публикация собранных данных в виде открытой базы вакансий без проверки условий использования источника.

7. Основные пользовательские сценарии

7.1. Создание поисковой конфигурации

Пользователь создаёт конфигурацию поиска:

```
{
  "name": "Python Backend Junior",
  "search_text": "Python FastAPI Django junior",
  "area_id": "1",
  "experience_id": "between1And3",
  "per_page": 50,
  "max_pages": 5,
  "is_active": true
}
```

Система сохраняет конфигурацию и использует её при запуске сбора вакансий.

7.2. Ручной запуск сбора вакансий

Пользователь запускает сбор вакансий через API или интерфейс.

Система:

1. получает активные поисковые конфигурации;
2. создаёт запись о запуске парсера;
3. отправляет запросы в HeadHunter API;
4. обрабатывает пагинацию;
5. сохраняет raw JSON;
6. нормализует данные;
7. создаёт или обновляет вакансии;
8. сохраняет статистику запуска.

7.3. Просмотр аналитики

Пользователь открывает аналитический endpoint или dashboard и видит:

- количество вакансий;
- новые вакансии за период;
- топ навыков;
- зарплатные вилки;
- распределение по городам;
- распределение по опыту;
- топ работодателей;
- динамику публикаций.

7.4. AI-фильтрация вакансий

Пользователь создаёт свой профиль:

```
{
  "target_roles": ["Junior Python Developer", "Technical Support Engineer", "System Administrator"],
  "skills": ["Python", "Linux", "Git", "Docker basics", "SQL basics"],
  "experience_summary": "Опыт" технического учёта в облачной инфраструктуре, работа с NetBox, взаимодействие с инженерами и ЦОД",
  "preferred_work_formats": ["remote", "hybrid"],
  "salary_min": 100000
}
```

Система сравнивает профиль с вакансиями и рассчитывает оценку.

7.5. Подготовка ручного отклика

Пользователь выбирает вакансию с высоким score.

Система генерирует черновик сопроводительного письма, но не отправляет отклик автоматически.

Пользователь:

1. читает объяснение;
2. проверяет риски;
3. редактирует письмо;
4. открывает вакансию на HeadHunter;
5. вручную отправляет отклик или пропускает вакансию.

8. Функциональные требования

8.1. Модуль HH API Client

Модуль должен отвечать за работу с HeadHunter API.

Функции:

- `search_vacancies(params)` - поиск вакансий;
- `get_vacancy(vacancy_id)` - получение детальной информации по вакансии;
- `get_areas()` - получение справочника регионов;
- `get_dictionaries()` - получение справочников;
- `validate_response(response)` - проверка ответа API.

Требования:

- использовать HTTPS;
- отправлять корректный HH-User-Agent;
- обрабатывать HTTP-ошибки;
- обрабатывать 429 Too Many Requests;
- поддерживать retry/backoff;
- логировать ошибки;
- возвращать JSON-данные в сервисный слой.

8.2. Модуль поисковых конфигураций

Система должна позволять:

- создавать поисковые конфигурации;
- просматривать список конфигураций;
- включать и отключать конфигурации;
- редактировать параметры поиска;
- ограничивать глубину сбора через `max_pages`.

Поля сущности `search_configs`:

```
id
name
search_text
area_id
experience_id
employment_id
schedule_id
professional_role_id
per_page
max_pages
is_active
created_at
updated_at
```

8.3. Модуль сбора вакансий

Модуль должен:

- получать активные поисковые конфигурации;
- создавать запись о запуске парсера;
- отправлять запросы к HH API;
- проходить страницы результата;
- сохранять raw JSON;
- передавать данные в `normalizer`;
- создавать новые вакансии;
- обновлять существующие вакансии;
- обновлять поле `last_seen_at`;
- фиксировать ошибки по отдельным вакансиям без остановки всего запуска.

8.4. Модуль хранения raw JSON

Система должна сохранять исходные ответы API.

Назначение:

- отладка ошибок;
- повторная нормализация данных без повторного запроса к API;
- анализ изменений структуры API;
- контроль качества парсинга.

Таблица `raw_vacancy_payloads`:

```
id
vacancy_hh_id
payload_json
fetched_at
```


8.5. Модуль нормализации

Модуль должен извлекать из raw JSON нормализованные поля:

```
hh_id
name
description
alternate_url
area_id
area_name
employer_id
employer_name
salary_from
salary_to
salary_currency
salary_gross
experience_id
experience_name
employment_id
employment_name
schedule_id
schedule_name
published_at
created_at_hh
archived
```

8.6. Модуль навыков

Система должна:

- сохранять навыки из вакансий;
- нормализовать написание навыков;
- связывать вакансии и навыки через many-to-many;
- считать частотность навыков.

Примеры нормализации:

```
postgres, Postgres, PostgreSQL -----> PostgreSQL
js, JavaScript -----> JavaScript
k8s, Kubernetes -----> Kubernetes
```

8.7. Модуль аналитики

Система должна рассчитывать:

- общее количество вакансий;
- количество вакансий по поисковым конфигурациям;
- количество новых вакансий за период;
- топ навыков;
- топ работодателей;
- распределение по городам;
- распределение по опыту;
- среднюю и медианную зарплату;
- динамику публикаций по датам.

8.8. Модуль профиля пользователя

Система должна позволять создать локальный профиль пользователя.

Поля user_profiles:

```
id
name
target_roles
skills
experience_summary
education
preferred_locations
preferred_work_formats
salary_min
salary_currency
created_at
updated_at
```

8.9. Модуль rule-based скоринга

Модуль должен рассчитывать базовый score вакансии без нейронной сети.

Пример формулы:

```
final_score =
    skill_score * 0.40
+ experience_score * 0.20
+ salary_score * 0.15
+ location_score * 0.10
+ role_score * 0.10
+ risk_score * 0.05
```

Интерпретация:

```
85-100 ----- сильное совпадение, стоит откликаться
70-84  ----- хорошее совпадение, стоит рассмотреть
50-69  ----- спорно, можно сохранить
0-49   ----- скорее не подходит
```

8.10. Модуль локальной AI-фильтрации

Модуль должен использовать локальную модель или локальный inference endpoint.

Задачи:

- сравнить профиль пользователя и вакансию;
- объяснить совпадения;
- выделить недостающие навыки;
- выделить риски;
- предложить действие: apply, maybe, skip;
- вернуть структурированный JSON.

Пример ответа:

```
{
  "match_score": 78,
  "decision": "maybe",
  "matched_skills": ["Python", "SQL", "Git"],
```

```
"missing_skills": ["Redis", "Kubernetes"],
"risks": Недостаточно[" коммерческого backendопыта-"],
"explanation": Вакансия подходит как переходная позиция, но требует усиления backend
навыков-.",
"recommended_action": Рассмотреть" и подготовить адаптированный отклик"
}
```

8.11. Модуль генерации черновика отклика

Модуль должен генерировать текст сопроводительного письма на основе:

- профиля пользователя;
- требований вакансии;
- совпавших навыков;
- недостающих навыков;
- опыта пользователя.

Система не должна отправлять письмо или отклик автоматически.

9. Нефункциональные требования

9.1. Надёжность

Система должна:

- продолжать работу при ошибке одной вакансии;
- логировать ошибки;
- не создавать дубли;
- корректно обрабатывать пустые зарплаты;
- корректно обрабатывать вакансии без навыков;
- поддерживать повторный запуск без порчи данных.

9.2. Производительность MVP

Для первой версии достаточно:

- до 10 поисковых конфигураций;
- до 50 000 вакансий в PostgreSQL;
- запуск сбора 1-2 раза в день;
- ответ простых API-запросов до 1 секунды;
- расчёт AI-score в фоне, а не в момент запроса списка вакансий.

9.3. Масштабируемость

Будущие расширения:

- Redis Queue;
- Celery worker;
- отдельный parser worker;
- отдельный AI worker;
- ClickHouse для аналитики;
- frontend dashboard;
- поддержка нескольких источников вакансий.

9.4. Безопасность

Система должна:

- хранить секреты в .env;
- не хранить токены в репозитории;
- не собирать персональные данные кандидатов;
- не использовать пользовательские cookies;
- не обходить ограничения API;
- ограничивать частоту запросов;
- иметь документ legal-boundaries.md.

9.5. Логирование

Логи должны фиксировать:

- старт сбора;
- завершение сбора;
- параметры поисковой конфигурации;
- количество найденных вакансий;
- количество созданных вакансий;
- количество обновлённых вакансий;
- количество ошибок;
- длительность запуска;
- ошибки HTTP;
- ошибки нормализации;
- ошибки AI-оценки.

10. Архитектура системы

10.1. MVP-архитектура

User

```
-----> FastAPI Backend
          -----> PostgreSQL
          -----> HH API Client
          -----> Parser Service
          -----> Analytics Service
          -----> AI Scoring Service
```

10.2. Архитектура с фоновыми задачами

Scheduler

```
-----> Parser Service
          -----> HH API
          -----> Raw JSON
          -----> Normalizer
          -----> PostgreSQL
```

AI Worker

```
-----> Vacancies without score
-----> User Profile
-----> Rule-based Scoring
-----> Local LLM
-----> Vacancy Scores
```

User

```
-----> FastAPI
          -----> Recommendations
          -----> Analytics
          -----> Drafts
```

11. Рекомендуемый стек

11.1. Backend

- Python 3.11+
- FastAPI
- Pydantic
- SQLAlchemy
- Alembic
- httpx
- APScheduler

11.2. База данных

- PostgreSQL для основного хранения.
- SQLite допустим только для раннего учебного прототипа.

11.3. AI-модуль

Варианты локального запуска:

- Ollama;
- llama.cpp;
- text-generation-webui;
- vLLM, если железо позволяет;
- отдельный локальный HTTP endpoint.

Возможные модели для локального теста:

- Qwen2.5 3B/7B Instruct в quantized-формате;
- Gemma 2 2B;
- Phi-3 Mini;
- Mistral 7B Instruct в 4-bit, если хватает ресурсов.

11.4. Инфраструктура

- Docker;
- Docker Compose;
- .env;
- GitHub Actions для тестов;
- README с инструкциями запуска.

12. Структура проекта

hh-vacancy-analytics/|—

```
backend/|
├── app/|
│   ├── main.py|
│   ├── api/|
│   │   ├── health.py|
│   │   ├── search_configs.py|
│   │   ├── parser.py|
│   │   ├── vacancies.py|
│   │   ├── analytics.py|
│   │   ├── profiles.py|
│   │   ├── recommendations.py|
│   │   └── drafts.py|
│   ├── core/|
│   │   ├── config.py|
│   │   ├── logging.py|
│   │   └── exceptions.py|
│   ├── db/|
│   │   ├── session.py|
│   │   ├── base.py|
│   │   └── migrations/|
│   ├── models/|
│   │   ├── vacancy.py|
│   │   ├── skill.py|
│   │   ├── search_config.py|
│   │   ├── parser_run.py|
│   │   ├── raw_payload.py|
│   │   ├── user_profile.py|
│   │   ├── vacancy_score.py|
│   │   ├── cover_letter_draft.py|
│   │   └── application_decision.py|
│   ├── schemas/|
│   │   ├── vacancy.py|
│   │   ├── search_config.py|
│   │   ├── analytics.py|
│   │   ├── profile.py|
│   │   └── recommendation.py|
│   ├── services/|
│   │   ├── hh_client.py|
│   │   ├── parser_service.py|
│   │   ├── normalizer.py|
│   │   ├── vacancy_service.py|
│   │   ├── analytics_service.py|
│   │   ├── scoring_service.py|
│   │   ├── embedding_service.py|
│   │   ├── local_llm_service.py|
│   │   ├── recommendation_service.py|
│   │   └── cover_letter_service.py|
│   └── prompts/|
│       ├── vacancy_scoring_prompt.txt|
│       └── cover_letter_prompt.txt|
├── tests/|
├── alembic.ini|
├── pyproject.toml|
└── Dockerfile|—
```

```
docs/|
|— technical-specification.md|
|— architecture.md|
|— database-schema.md|
|— api-contract.md|
|— ai-scoring.md|
|— legal-boundaries.md| |—
docker-compose.yml|—
.env.example|—
README.md|—
Makefile
```

13. Модель данных

13.1. search_configs

```
id
name
search_text
area_id
experience_id
employment_id
schedule_id
professional_role_id
per_page
max_pages
is_active
created_at
updated_at
```

13.2. vacancies

```
id
hh_id
name
description
alternate_url
area_id
area_name
employer_id
employer_name
salary_from
salary_to
salary_currency
salary_gross
experience_id
experience_name
employment_id
employment_name
schedule_id
schedule_name
published_at
created_at_hh
archived
```

```
first_seen_at
last_seen_at
updated_at
```

13.3. skills

```
id
name
normalized_name
created_at
```

13.4. vacancy_skills

```
vacancy_id
skill_id
```

13.5. parser_runs

```
id
search_config_id
status
started_at
finished_at
found_count
created_count
updated_count
failed_count
error_message
duration_sec
```

13.6. raw_vacancy_payloads

```
id
vacancy_hh_id
payload_json
fetched_at
source
```

13.7. user_profiles

```
id
name
target_roles
skills
experience_summary
education
preferred_locations
preferred_work_formats
salary_min
salary_currency
created_at
updated_at
```


13.8. vacancy_scores

```
id
vacancy_id
user_profile_id
rule_score
embedding_score
llm_score
final_score
decision
matched_skills
missing_skills
risks
explanation
recommended_action
created_at
updated_at
```

13.9. cover_letter_drafts

```
id
vacancy_id
user_profile_id
draft_text
status
created_at
updated_at
```

13.10. application_decisions

```
id
vacancy_id
user_profile_id
decision
reason
decided_at
```

14. REST API

14.1. Health

```
GET /health
```

ОТВЕТ:

```
{
  "status": "ok"
}
```

14.2. Search configs

```
GET    /search-configs
POST   /search-configs
GET    /search-configs/{id}
PATCH /search-configs/{id}
DELETE /search-configs/{id}
```

14.3. Parser

```
POST /parser/run
GET  /parser/runs
GET  /parser/runs/{id}
```

14.4. Vacancies

```
GET /vacancies
GET /vacancies/{id}
```

Фильтры:

```
skill
area
employer
salary_from
salary_to
experience
published_from
published_to
limit
offset
```

14.5. Analytics

```
GET /analytics/summary
GET /analytics/skills
GET /analytics/salaries
GET /analytics/employers
GET /analytics/areas
GET /analytics/dynamics
```

14.6. Profiles

```
GET    /profiles
POST   /profiles
GET    /profiles/{id}
PATCH /profiles/{id}
DELETE /profiles/{id}
```

14.7. Recommendations

```
POST /recommendations/calculate
GET  /recommendations?vacancy_score_min=70
GET  /recommendations/{vacancy_id}
```

14.8. Drafts

```
POST /drafts/cover-letter
GET /drafts
GET /drafts/{id}
PATCH /drafts/{id}
```

15. Алгоритм сбора вакансий

1. Получить активные `search_configs`.
2. Для каждой конфигурации создать `parser_run`.
3. Запросить первую страницу HH API.
4. Определить количество страниц.
5. Последовательно пройти страницы до `max_pages`.
6. Для каждой вакансии:
 - 6.1. Сохранить raw JSON.
 - 6.2. Нормализовать данные.
 - 6.3. Проверить наличие вакансии по `hh_id`.
 - 6.4. Если вакансии нет - создать.
 - 6.5. Если вакансия есть - обновить.
 - 6.6. Обновить `skills`.
 - 6.7. Обновить `last_seen_at`.
7. Завершить `parser_run`.
8. Сохранить статистику запуска.

16. Алгоритм рекомендации вакансий

1. Получить профиль пользователя.
2. Получить вакансии без `score` или вакансии, требующие пересчёта.
3. Посчитать `skill_score`.
4. Посчитать `experience_score`.
5. Посчитать `salary_score`.
6. Посчитать `location_score`.
7. Посчитать `role_score`.
8. Сформировать `rule_score`.
9. При необходимости посчитать `embedding_score`.
10. Передать краткую карточку вакансии в локальную LLM.
11. Получить `explanation`, `risks`, `recommended_action`.
12. Рассчитать `final_score`.
13. Сохранить результат в `vacancy_scores`.

17. Критерии приёмки MVP

MVP считается готовым, если:

1. Проект запускается через Docker Compose.
2. Можно создать поисковую конфигурацию.
3. Можно запустить сбор вакансий вручную.
4. Вакансии сохраняются в PostgreSQL.
5. Повторный запуск не создаёт дубли.
6. Raw JSON сохраняется отдельно.
7. Навыки сохраняются в отдельную таблицу.

8. Есть API для просмотра вакансий.
9. Есть API для базовой аналитики.
10. Есть профиль пользователя.
11. Есть rule-based score вакансии.
12. Есть AI-оценка или заглушка AI-модуля с тем же контрактом.
13. Есть генерация черновика сопроводительного письма.
14. Отклик не отправляется автоматически.
15. Есть документация по архитектуре и ограничениям.
16. Есть README с запуском проекта.
17. Есть базовые тесты для HH client, normalizer и scoring service.

18. Этапы реализации

Этап 1. Минимальный HH API client

Результат:

- скрипт получает вакансии по запросу;
- выводит название, компанию, город, зарплату и ссылку;
- корректно обрабатывает HTTP-ошибки.

Этап 2. Сохранение raw JSON

Результат:

- вакансии сохраняются в JSON-файл;
- структура ответа изучена;
- выделены нужные поля.

Этап 3. PostgreSQL и модели

Результат:

- создана схема БД;
- настроены SQLAlchemy и Alembic;
- вакансии сохраняются в БД.

Этап 4. Дедупликация

Результат:

- hh_id является уникальным;
- повторный запуск обновляет существующие вакансии.

Этап 5. FastAPI

Результат:

- есть endpoints для вакансий, конфигураций и запуска парсера.

Этап 6. Аналитика

Результат:

- топ навыков;

- зарплаты;
- города;
- работодатели;
- динамика публикаций.

Этап 7. Профиль пользователя и rule-based scoring

Результат:

- пользователь создаёт профиль;
- система считает score вакансий.

Этап 8. Локальная AI-фильтрация

Результат:

- локальная модель или заглушка возвращает структурированную оценку;
- результат сохраняется в БД.

Этап 9. Черновики откликов

Результат:

- система генерирует письмо;
- пользователь вручную принимает решение.

Этап 10. Docker и документация

Результат:

- запуск одной командой;
- оформленный README;
- документация в docs/.

19. Риски и меры снижения

Риск	Мера снижения
Изменится структура API	Хранить raw JSON, покрыть normalizer тестами
API ограничит частоту запросов	Retry/backoff, лимиты, расписание
Дубли вакансий	Уникальный ключ по hh_id
Не указана зарплата	Поддерживать NULL и считать статистику аккуратно
Разные написания навыков	Нормализация навыков
AI даёт неверные рекомендации	Показывать объяснения и оставлять решение человеку
Юридические ограничения	Не автоматизировать отклики, не собирать резюме, использовать API
Слишком большой объём данных	Ограничить max_pages, добавить фоновые задачи

20. Документы проекта

В репозитории должны быть следующие документы:

```
README.md
README_RU.md
docs/technical-specification.md
docs/architecture.md
docs/database-schema.md
docs/api-contract.md
docs/ai-scoring.md
docs/legal-boundaries.md
docs/development-roadmap.md
```

21. Формулировка для резюме

Разработал платформу для сбора и анализа открытых вакансий HeadHunter через официальный API. Реализовал ETL-процесс: получение вакансий, хранение raw JSON, нормализация данных, дедупликация и сохранение в PostgreSQL.

Добавил модуль интеллектуальной фильтрации вакансий: система сравнивает требования вакансии с профилем пользователя, рассчитывает score релевантности, выявляет совпадающие и недостающие навыки, объясняет результат и формирует черновик сопроводительного письма.

Проект включает FastAPI, PostgreSQL, SQLAlchemy, Alembic, Docker Compose, планировщик задач, REST API, аналитику по рынку вакансий и локальную LLM/embedding-модель для рекомендаций.

22. Источники

1. Официальная документация HeadHunter API
2. Репозиторий документации HeadHunter API
3. Документация по вакансиям в HeadHunter API
4. Документация по ошибкам HeadHunter API
5. Пользовательское соглашение HeadHunter